Ensemble Learning and Random Forests

Anca Ralescu

Voting Classifiers

Suppose one has trained several each one achieving about 80% accuracy, as shown in Figure 1.

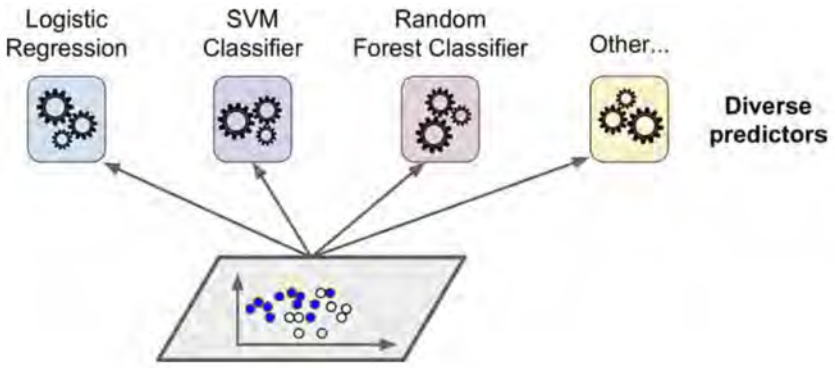


Figure 1: Training diverse classifiers.

We can aggregate them by a (hard) voting procedure as shown in Figure 2. The hard voting classifier often achieves a higher accuracy than the best classifier in the ensemble.

In fact, even if each classifier is a weak learner (i.e., it does only slightly better than random guessing), the ensemble can still be a strong learner (achieving high accuracy), provided that

- there are a sufficient number of weak learners
- they are sufficiently diverse

Explanation of a seemingly strange phenomenon

Consider a slightly biased coin, C, such that

$$P(C = Head) = 0.51, \text{ and } P(C = tail) = 0.49$$

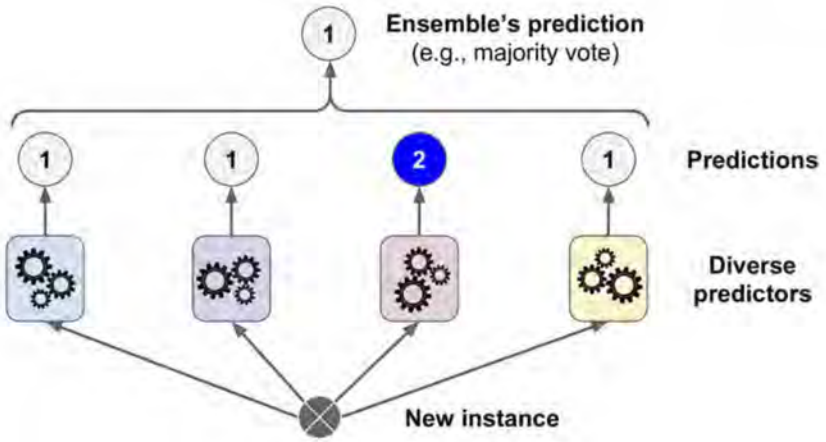


Figure 2: Hard voting classifier predictions

Let's denote "Head" by 1, and "Tail" by 0. Then we have that

$$P(C = 1) = 0.51, \text{ and } P(C = 0) = 0.49$$

and hence the mean (expected value) of C is

$$E(C) = 1 \times P(C = 1) + o \times P(C = 0) = 0.51$$

Toss C 1,000 times: the outcome is ≈ 510 heads and ≈ 490 tails.

Computing, we have

$$P(\#Heads > \#Tails \text{ after 1,000 tosses}) \approx 0.75$$

Similarly,

$$P(\#Heads > \#Tails \text{ after 10,000 tosses}) \approx 0.97$$

This behavior is the result of the Law of Large Numbers from Probability Theory, which in this case becomes that *as the number of tosses increases, the ratio*

$$\frac{\#heads}{\#tosses}$$

approaches $P(C = Heads) = 0.51$.

Figure 3 shows 10 series of biased coin tosses: it can be seen that as the number of tosses increases, the ratio of heads approaches 0.51. Eventually all 10 series end up so close to 0.51, that they are consistently above 0.50.

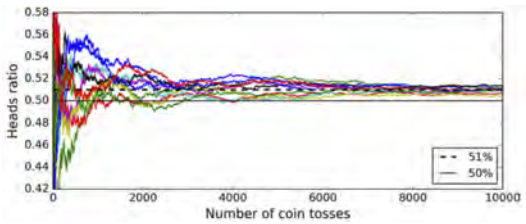


Figure 3: The law of large numbers

Now, suppose we replace the coin by a classifier. Each classifier acts like a coin, either correctly classifies (i.e. comes Heads up), or incorrectly classifies, (i.e., comes Tails up). Assume that each classifiers has 0.51 probability of correct classification.

Consider an ensemble of classifiers containing 1,000 classifiers that are individually correct with probability of only 0.51 (i.e. barely better than random guessing). The majority voted classifier, can only hope for up to 0.75 accuracy!

However, unlike for the coin toss experiment, where successive toss outcomes are **independent**, in the case of these classifiers **we cannot assume independence**, because, after all, *they are all trained on the same data set*.
Because of lack of independence, these classifiers are likely to make the same types of errors, so there will be many majority votes for the wrong class, reducing the ensemble's accuracy.

When the classifiers estimate class probabilities as opposed to hard class labels, then we have *a soft voting classifier.*

Very Important Note 1 *Ensemble methods work best when the predictors are as independent from one another as possible.*

How to get diverse classifiers:

- *Train them using very different algorithms*

- *Use different training data.*

This increases the chance that they will make very different types of errors, improving the ensemble's accuracy.

Bagging and Pasting

To obtain diverse classifiers, use the same training algorithm for every predictor, but different random subsets of the training set.

- Bagging (introduced by Breiman[1], in *Bagging Predictors*, 1996): Sampling with replacement (the data sampled is put back into the the set). Name comes from *bootstrap aggregating*.
  - allows training instances to be sampled several times across multiple predictors,
  - allows training instances to be sampled several times for the same predictor.
- Pasting (introduced in *Pasting small votes for classification in large databases and on-line*, L. Breiman (1999)): Sampling is performed without replacement.
  - allows training instances to be sampled several times across multiple predictors
  - DOES NOT allow training instances to be sampled several times for the same predictor.
- Both training and prediction can be made in parallel.

Sampling and training process is represented in Figure 4.

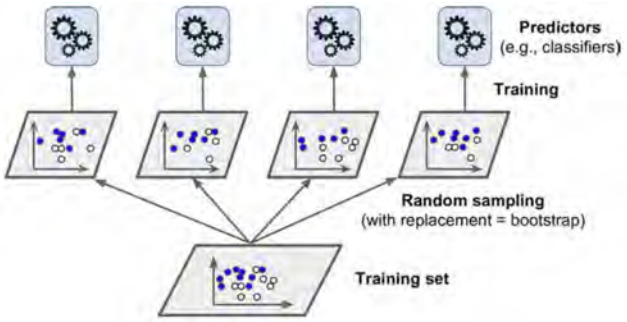


Figure 4: Pasting/bagging training set sampling and training.

[1] https://en.wikipedia.org/wiki/Leo_Breiman

Aggregation

Once all predictors are trained, the ensemble can make a prediction for a new instance by simply aggregating the predictions of all predictors.

- Typically, the aggregation function is the *statistical mode* (i.e., the most frequent prediction, just like a hard voting classifier) for classification, or the average for regression.
- Each individual predictor has a higher bias than if it were trained on the original training set, but aggregation reduces both bias and variance. Generally, the net result is that the ensemble has a similar bias but a lower variance than a single predictor trained on the original training set.

Figure 5 illustrates bagging versus single algorithm for Decision Trees.

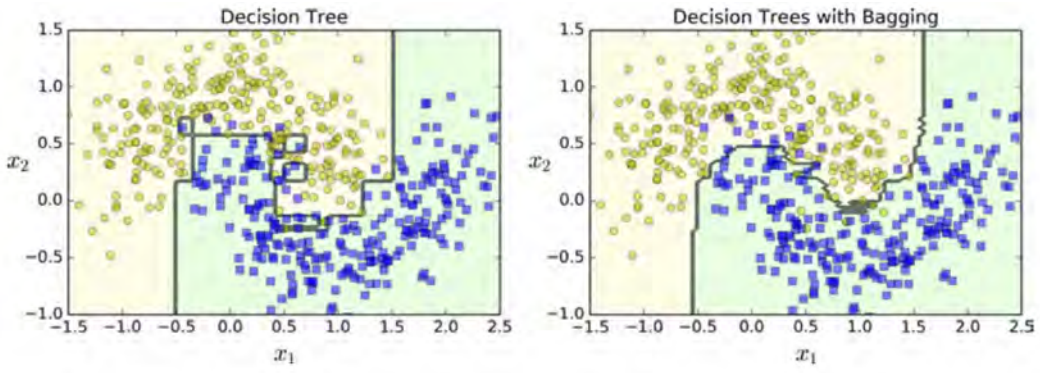


Figure 5: single Decision Tree versus a bagging ensemble of 500 trees

- Bootstrapping introduces a bit more diversity in the subsets that each predictor is trained on, so bagging ends up with a slightly higher bias than pasting
  - Predictors end up being less correlated so the ensemble's variance is reduced.
- Overall, bagging often results in better models, which explains why it is generally preferred.
- If time and CPU power are available, one can use cross-validation to evaluate both bagging and pasting and select the one that works best.

Random Patches and Random Subspaces

- Random patches: Random sampling of features and training data.
- Random Subspaces: Random sampling of features

Extremely Randomized Trees: Extra-Trees:

- Introduce further randomness by using *random thresholds for each feature, rather than searching for the best thresholds*.
- Much faster to train

Random Forests

- Ensemble Decision Trees trained via the bagging method (or, sometimes, pasting).
- The maximum number of sample is usually the whole training set.
- The Random Forest algorithm introduces extra randomness when growing trees.
  - Instead of searching for the very best feature when splitting a node, it searches for the best feature among a random subset of features.
  - The resulting tree has greater diversity: trades a higher bias for a lower variance
  - Yields an overall better model.
  - It is hard to predict which, a RandomForestClassifier or an ExtraTreesClassifier performs better.
    - Try both and compare on cross-validation
    - Tune the hyperparameters using grid search (evaluating all the possible combinations of hyperparameters on cross-validation).

Feature Importance

In a DT, important[2] features are likely to appear closer to the root of the tree, while unimportant features will often appear closer to the leaves (or not at all).

[2]But recall that this is a vague concept: information gain, or other criteria come into play, but also, the cost associated to feature.

Use the *average depth* at which a feature appears across all trees in the forest to decide the feature importance.

For example, it seems that in order of importance, features for the iris dataset are:

- petal length (0.441)
- petal width (0.423)
- sepal length (0.112)
- sepal width (0.023)

For the MNIST[3] data set, pixel importance is illustrated in Figure 6

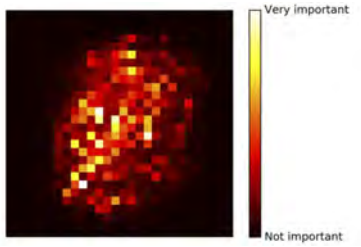


Figure 6: MNIST pixel importance (according to a Random Forest classifier)

Boosting (hypothesis boosting)

Any Ensemble method that can combine several weak learners into a strong learner.

Idea: Train predictors sequentially, each trying to correct its predecessor.

Most popular:

- Adaptive Boosting: AdaBoost
- Gradient Boosting

AdaBoost

- New predictor corrects its predecessor by paying more attention to the training instances that the predecessor *underfitted.*

[3]MNIST data set is a set of 60000 handwritten characters.

- New predictors are focusing more and more on the hard cases.

For example:

1. Train a first base classifier (such as a Decision Tree)
2. Make predictions on the training set.
3. Increase relative weight of misclassified training instances
4. Train next classifier using the updated weights
    - make predictions on the training set,
    - Update weights

Figure 7 illustrates this process.

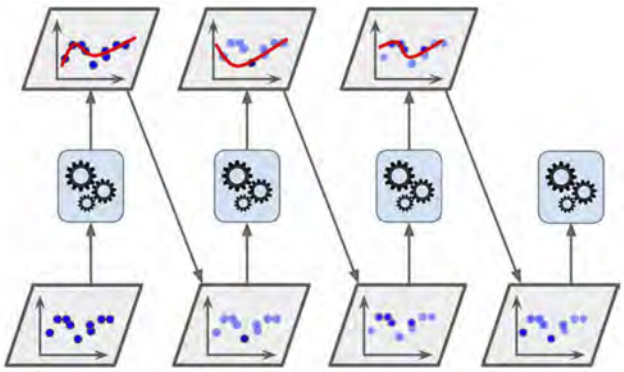


Figure 7: AdaBoost sequential training with instance weight updates

Figure 8 shows the decision boundaries of five consecutive predictors on the moons dataset :

- The first classifier gets many instances wrong, so their weights get boosted.
- The second classifier therefore does a better job on these instances, and so on.
- The plot on the right represents the same sequence of predictors except that the learning rate is halved (i.e., the misclassified instance weights are boosted half as much at every iteration).

Very Important Note 2 *Note the following:*

- *This sequential learning technique has some similarities with Gradient Descent, except that instead of tweaking a single predictor's parameters to minimize a cost function, AdaBoost adds predictors to the ensemble, gradually making it better.*
- *Once all predictors are trained, the ensemble makes predictions very much like bagging or pasting, except that predictors have different weights depending on their overall accuracy on the weighted training set.*
- *Drawback:sequential learning techniques cannot be parallelized (or only partially), since each predictor can only be trained after the previous predictor has been trained and evaluated. As a result, it does not scale as well as bagging or pasting.*

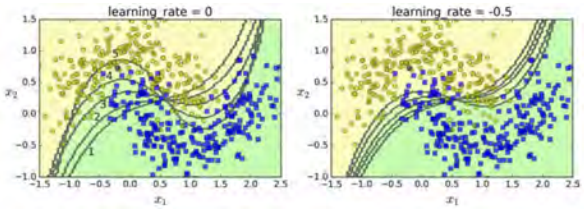


Figure 8: Decision boundaries of consecutive predictors (a kind of polynomial regression).

The AdaBoost Algorithm

Define the cumulative weight of errors for predictor j for instance i

$$Numerator_j = \sum_{i=1, \hat{y}_j^{(i)} \neq y^{(i)}}^{m} w_{(i)}$$

and the total weights

$$Denominator = \sum_{i=1}^{m} w_{(i)}$$

Then the error rate for the jth predictor is

$$r_j = \frac{Numerator_j}{Denominator}$$

Initially,

$$w^{(i)} = \frac{1}{m}.$$

Let α_j denote the weight of the jth predictor, and η the learning rate (initially set equal to 1). Then α_j is updated as follows:

$$\alpha_j = \eta \frac{1 - r_j}{r_j}.$$

Using α_j the instance weights are updated according to the following rule

for $i = 1, \ldots, m$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \widehat{y}_j^{(i)} = y^{(i)}, \text{ i.e. if the prediction is correct there is no change} \\ w^{(i)} e^{(\alpha_j)} & \text{if } \widehat{y}_j^{(i)} \neq y^{(i)}, \text{ i.e. update weight if the prediction is incorrect} \end{cases}$$

Reormalize weights such that they will add up to 1: $w^{(i)} \leftarrow \frac{w^{(i)}}{\sum_{k=1}^{m} w^{(k)}}$

AdaBoost Predictions

For each instance $\mathbf{x}$, predict

$$\widehat{y}(\mathbf{x}) = \arg\max_{k} \sum_{j=1,\, \widehat{y}_j(\mathbf{x})=k}^{N} \alpha_j, \text{ where } N \text{ is the number of predictors}$$

That is, the predicted class is the one that receives the majority of weighted votes.

Gradient Boosting

- Works sequentially.
- It Finds a new predictor to the *residual errors made by the previous predictor*

Example 1 *(Gradient Tree Boosting, or Gradient Boosted Regression Trees (GBRT)*

1. *Fit a decision tree to a training set (e.g., a noisy quadratic training set):* $pred(1)$

2. *Train a 2nd DT on the residual errors of the 1st predictor,* $pred(2)$
3. *Train a 3rd DT on the residual errors of the 2nd predictor,* $pred(3)$
4. *Result: Ensemble containing three threes, which makes* $pred = \sum_{i=1}^{3} pred(i)$,

This process is illustrated in Figure 9.

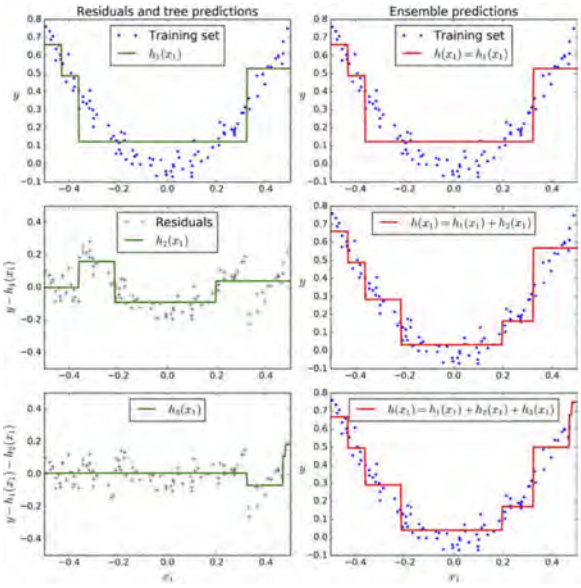


Figure 9: Gradient Boosting

An issue that arises in GBRT trees is knowing when to stop. Figure 10 illustrates the two extremes, when there are not enough predictors, and when there are too many.

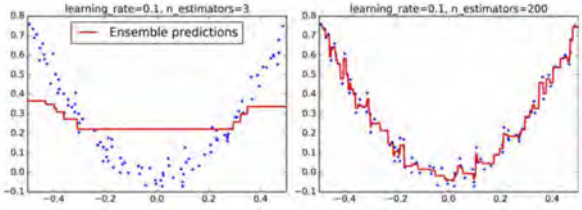


Figure 10: GBRT ensembles with not enough predictors (left) and too many (right)

One solution, is to implement an early stopping approach and use a validation set. Based on the validation result then retrain the (optimal) number of trees. Figure 11 illustrates this approach.

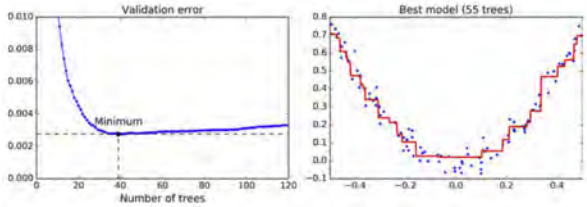


Figure 11: Tuning the number of trees using early stopping